



Lecture 5: TurboVec in RAG Pipeline

Using the Rust-Powered Vector Index for Fast, Memory-Efficient RAG

Part of the RAG Mastery Series



What You Will Learn Today

1. What is TurboVec and why it's special
 2. How TurboVec differs from FAISS, Qdrant, etc.
 3. Installing and setting up TurboVec
 4. Building a RAG pipeline with TurboVec
 5. TurboVec + LangChain integration
 6. Performance benchmarks and when to use TurboVec
-

1. What is TurboVec?

TurboVec is a Rust-based vector index with Python bindings, developed by Ryan Codrai. It uses **Google TurboQuant** (ICLR 2026) for extreme memory efficiency.

Key Features

Feature	TurboVec	FAISS	Qdrant
Language	Rust + Python	C++ + Python	Rust
RAM Usage	16x less than FAISS	Baseline	Moderate
Speed	Faster than FAISS	Fast	Fast
Training Required	No	Some indexes need it	No
Filtered Search	Yes	Limited	Yes

Integrations	LangChain, LlamaIndex, Haystack, LangChain, Agno	Many	Many
License	Open source	MIT	Apache 2.0

The TurboQuant Advantage

TurboQuant is a vector quantization technique from Google (published at ICLR 2026) that:

- Compresses vectors to **4-bit or 8-bit** representations
- Achieves **16x RAM reduction** with minimal accuracy loss
- Requires **no training** (works out of the box)
- Enables **filtered search** on compressed vectors

Standard vector (float32): 1536 dimensions × 4 bytes = 6,144 bytes per vector

TurboQuant (4-bit): 1536 dimensions × 0.5 bytes = 768 bytes per vector
→ 8x compression!

With additional optimizations: up to 16x total RAM reduction

Why This Matters for RAG

Scenario: 1 million document chunks, 1536-dim embeddings

Standard (float32): 1M × 6,144 bytes = 6.1 GB RAM

TurboVec (4-bit): 1M × 384 bytes = 0.38 GB RAM
(16x less memory!)

→ You can run a 1M document RAG system on a laptop with 8GB RAM

→ No need for expensive cloud instances

→ Faster search due to better cache utilization

2. Installing TurboVec

Prerequisites

- Python 3.8+
- pip

Installation

```
pip install turbovec
```

Verify Installation

```
import turbovec
print(turbovec.__version__)

# Check available backends
print(turbovec.available_backends())
```

3. TurboVec Basics

Creating an Index

```
import turbovec
import numpy as np

# Define dimensions
dim = 384 # Must match your embedding model's output dimension

# Create an index
index = turbovec.Index(
    dim=dim,
    metric="cosine",      # or "euclidean", "dot"
    quantization="turboquant_4bit", # Key feature!
    max_elements=1_000_000
)
```

Adding Vectors

```

import numpy as np

# Generate sample embeddings (in practice, from your embedding model)
num_vectors = 10000
embeddings = np.random.randn(num_vectors, 384).astype(np.float32)

# Add vectors with IDs
ids = np.arange(num_vectors)
index.add(embeddings, ids)

# You can also add metadata
metadata = [
    {"doc_id": "doc_1", "chunk_id": "chunk_0", "text": "..."},
    {"doc_id": "doc_1", "chunk_id": "chunk_1", "text": "..."},
    # ...
]

```

Searching

```

# Single query
query_vector = np.random.randn(384).astype(np.float32)
distances, ids = index.search(query_vector, k=10)

# Batch query
query_vectors = np.random.randn(5, 384).astype(np.float32)
all_distances, all_ids = index.search(query_vectors, k=10)

print(f"Top 10 results: {all_ids[0]}")
print(f"Distances: {all_distances[0]}")

```

Filtered Search (Unique Feature!)

```
# Search only within specific document IDs
filter_fn = lambda meta: meta["doc_id"] == "doc_1"
distances, ids = index.search(
    query_vector,
    k=10,
    filter=filter_fn
)

# Search within a date range
filter_fn = lambda meta: meta["year"] == 2025
distances, ids = index.search(query_vector, k=10, filter=filter_fn)
```

4. Building a Complete RAG Pipeline with TurboVec

Full Implementation

```

import turbovec
import numpy as np
from sentence_transformers import SentenceTransformer

class TurboVecRAG:
    def __init__(self, embedding_model="all-MiniLM-L6-v2"):
        self.encoder = SentenceTransformer(embedding_model)
        self.dim = self.encoder.get_sentence_embedding_dimension()

        # Create TurboVec index with 4-bit quantization
        self.index = turbovec.Index(
            dim=self.dim,
            metric="cosine",
            quantization="turboquant_4bit",
            max_elements=1_000_000
        )

        self.documents = {} # id -> text mapping
        self.metadata = {} # id -> metadata mapping

    def add_documents(self, texts: list[str], metadatas: list[dict] =
None):
        """Add documents to the RAG index."""
        # Generate embeddings
        embeddings = self.encoder.encode(texts, show_progress_bar=True)
        embeddings = embeddings.astype(np.float32)

        # Generate IDs
        start_id = len(self.documents)
        ids = np.arange(start_id, start_id + len(texts))

        # Store documents and metadata
        for i, text in enumerate(texts):
            self.documents[ids[i]] = text
            if metadatas:
                self.metadata[ids[i]] = metadatas[i]

```

```

        # Add to TurboVec index
        self.index.add(embeddings, ids)

        print(f"Added {len(texts)} documents. Total:
{len(self.documents)}")

    def search(self, query: str, k: int = 5, filter_dict: dict = None):
        """Search for relevant documents."""
        # Encode query
        query_embedding = self.encoder.encode([query])
        query_embedding = query_embedding.astype(np.float32)

        # Create filter function if needed
        filter_fn = None
        if filter_dict:
            def filter_fn(meta):
                return all(meta.get(k) == v for k, v in
filter_dict.items())

        # Search
        distances, ids = self.index.search(query_embedding, k=k,
filter=filter_fn)

        # Format results
        results = []
        for dist, idx in zip(distances[0], ids[0]):
            results.append({
                "id": int(idx),
                "score": float(dist),
                "text": self.documents.get(int(idx), ""),
                "metadata": self.metadata.get(int(idx), {})
            })

        return results

    def query(self, question: str, llm_client, k: int = 5):
        """Full RAG query: retrieve + generate."""

```

```
# Step 1: Retrieve relevant documents
results = self.search(question, k=k)

# Step 2: Build context
context = "\n\n".join([r["text"] for r in results])

# Step 3: Build prompt
prompt = f"""\
Use the following context to answer the question.
If the context doesn't contain the answer, say "I don't know."

Context:
{context}

Question: {question}

Answer: ""
```

```
# Step 4: Generate answer
answer = llm_client.generate(prompt)

return {
    "answer": answer,
    "sources": results
}
```

```
# Usage Example
```

```
rag = TurboVecRAG()
```

```
# Add documents
```

```
documents = [
    "Machine learning is a subset of artificial intelligence...",
    "Deep learning uses neural networks with multiple layers...",
    "Natural language processing deals with text understanding...",
    "Computer vision enables machines to understand images...",
    "Reinforcement learning trains agents through rewards...",
]
```

```
metadatas = [
    {"topic": "ML", "source": "textbook_ch1"},
    {"topic": "DL", "source": "textbook_ch2"},
    {"topic": "NLP", "source": "textbook_ch3"},
    {"topic": "CV", "source": "textbook_ch4"},
    {"topic": "RL", "source": "textbook_ch5"},
]

rag.add_documents(documents, metadatas)

# Search
results = rag.search("What is deep learning?", k=3)
for r in results:
    print(f"Score: {r['score']:.3f} | {r['text'][:80]}...")

# Filtered search (only ML topic)
results = rag.search("learning algorithms", k=3, filter_dict={"topic":
"ML"})
```

5. TurboVec + LangChain Integration

```
from langchain.schema import BaseRetriever, Document
from langchain.chains import RetrievalQA
from langchain_community.llms import Ollama

class TurboVecRetriever(BaseRetriever):
    """Custom LangChain retriever using TurboVec."""

    def __init__(self, turbovec_rag, k=5):
        self.rag = turbovec_rag
        self.k = k

    def _get_relevant_documents(self, query):
        results = self.rag.search(query, k=self.k)
        return [
            Document(
                page_content=r["text"],
                metadata=r["metadata"]
            )
            for r in results
        ]

# Use with LangChain QA chain
retriever = TurboVecRetriever(rag, k=5)
llm = Ollama(model="llama3")

qa_chain = RetrievalQA.from_chain_type(
    llm=llm,
    retriever=retriever,
    return_source_documents=True
)

result = qa_chain.invoke({"query": "What is deep learning?"})
print(result["result"])
print("Sources:", result["source_documents"])
```

6. TurboVec + LlamaIndex Integration

```
from llama_index.core import VectorStoreQuery, QueryBundle
from llama_index.core.schema import TextNode

# Create LlamaIndex nodes with TurboVec embeddings
nodes = []
for i, (text, meta) in enumerate(zip(documents, metadatas)):
    node = TextNode(
        text=text,
        metadata=meta,
        embedding=embeddings[i].tolist()
    )
    nodes.append(node)

# Query using TurboVec index
query_bundle = QueryBundle(query_str="What is NLP?")
query_embedding = encoder.encode([query_bundle.query_str])

distances, ids = index.search(query_embedding.astype(np.float32), k=5)

# Get the matching nodes
retrieved_nodes = [nodes[i] for i in ids[0]]
```

7. Performance Comparison

Memory Usage (1M vectors, 1536 dimensions)

System	RAM Usage	Relative
FAISS (float32)	6.1 GB	1x (baseline)
FAISS (PQ)	0.8 GB	7.6x less
Qdrant	6.5 GB	~1x
TurboVec (4-bit)	0.38 GB	16x less

Search Speed (1M vectors, top-10)

System	Latency (ms)	QPS
FAISS (flat)	45	22,000
FAISS (HNSW)	2.1	476,000
Qdrant	1.8	555,000
TurboVec	1.5	666,000

Accuracy (Recall@10 on standard benchmarks)

System	Recall@10
FAISS (flat, exact)	1.000
FAISS (HNSW)	0.985
FAISS (PQ)	0.940
TurboVec (4-bit)	0.975
TurboVec (8-bit)	0.992

8. When to Use TurboVec

✓ Use TurboVec When:

- Memory is limited (laptop, edge devices, cost-sensitive)
- You have millions of vectors but limited RAM
- You need filtered search (search within subsets)
- You want fast search without complex infrastructure
- Building on-device RAG (mobile, IoT)

✗ Don't Use TurboVec When:

- You need a full database (persistence, replication, multi-user)
- You need real-time updates with concurrent writes

- You need advanced features like hybrid search built-in
- Your dataset is small (< 100K vectors) — FAISS is fine

The Sweet Spot

TurboVec is perfect for:

- Single-application RAG systems
- Memory-constrained environments
- Edge deployment
- Cost-sensitive production systems
- Prototyping with large datasets on a laptop

9. Quick Summary

TurboVec: Rust-based vector index with Google TurboQuant compression

Key Benefit: 16x less RAM than FAISS with minimal accuracy loss

No Training: Works out of the box

Filtered Search: Search within subsets without separate indexes

Integrations: LangChain, LlamaIndex, Haystack, Agno

Best For: Memory-constrained RAG, edge deployment, cost-sensitive systems



Homework

1. Install TurboVec: `pip install turbovec`
2. Create an index with 1000 document embeddings
3. Compare memory usage with FAISS for the same data
4. Implement filtered search (filter by category)
5. Build a complete RAG system: TurboVec + any free LLM (Ollama)

Next Lecture: Building a Production RAG Project — End-to-end system design